CUDA Reduce 算子优化笔记

> 参考资料：AIInfraGuide《3.1 CUDA Reduce 算子优化》
> 代码：本目录 `Operator/Reduce/`（V0 → V8，每个版本一个 `.cu`，文件头注释记录了实测跑分）
> 实测条件：`N =` 2^{27} `= 134217728`（128M 个 float，512 MiB，全为 1.0），`Block = 256`，`Warmup = 5`，`Iterations = 100`

一、问题定义与性能模型

Reduce 是「多输入 → 单输出」的一类操作：

- Sum Reduce：$\sum x_i$
- Max / Min Reduce
- Dot Product：$\sum a_i \times b_i$

本文讨论 Sum Reduce：输入长度 `N` 的 float 数组，输出所有元素之和。

每个元素只做 1 次 load + 1 次 add，因此

- 算术强度 = 1 FLOP / 4 Byte = **0.25 FLOP/Byte**
- 属于典型的 **Memory-Bound（访存密集型）**
- 优化目标是**把有效带宽打满**，而不是减少计算量

```
// 每版 main() 里统一使用这个口径评估
double bandwidth = (double)size / (milliseconds * 1e6);   // GB/s，size = N * sizeof(float)
```

Reduce 的四类瓶颈与对应解法：

瓶颈	版本解法
Warp Divergence（计算效率低）	V1 交错寻址、V2 步长反转
Bank Conflict（Shared Memory 冲突）	V2 步长反转
多余 `__syncthreads()` / 循环开销	V4 展开最后一个 Warp、V5 模板全展开

瓶颈	版本解法
访存效率低 / GPU 利用不足	V3 每线程双元素、V6 Warp Shuffle、V7 Grid-Stride Loop、V8 float4

二、版本演进总览（本机实测）

版本	核心优化点	Grid	时间 (ms)	带宽 (GB/s)	相对 V0
V0	朴素树形规约（步长递增）	524288	5.776	92.95	1.00x
V1	交错寻址，减少 Warp Divergence	524288	4.171	128.71	1.39x
V2	步长反转，消除 Bank Conflict	524288	3.341	160.68	1.73x
V3	每线程处理 2 个元素	262144	1.742	308.20	3.32x
V3 + stride loop	同上 + Grid-Stride Loop	1024	0.928	578.47	6.22x
V4	展开最后一个 Warp	262144	1.179	455.22	4.90x
V5	模板参数，完全循环展开	262144	1.143	469.50	5.05x
V6	Warp Shuffle 两级规约	262144	1.027	522.54	5.62x
V7	Grid-Stride Loop + Warp Shuffle	1024	0.923	**581.67**	**6.26x**
V8	V7 + float4 向量化加载	272	0.929	578.11	6.22x

V7 的 Grid 扫描（`Test_V7.cu`，Block = 256）：

Grid	128	256	512	1024	2048	4096	8192
GB/s	545.26	575.60	578.04	579.94	580.35	578.37	580.29

> Grid ≥ 512 之后进入平台期（~578–581 GB/s），说明此时瓶颈已经是显存带宽而不是 Grid 规模。

三、各版本详解

V0：朴素树形规约 —— Warp Divergence

```
// V0: 朴素的树形规约（步长从小到大）
// 性能问题：tid % (2 * stride) == 0
// 导致 Warp 内出现大量分化 -----> Warp Divergence
__global__ void reduce_naive(const float* in, float* out, int N){
    extern __shared__ float smem[];

    int gid = blockDim.x * blockIdx.x + threadIdx.x;
    int tid = threadIdx.x;

    smem[tid] = (gid < N) ? in[gid] : 0.0f;
    __syncthreads();

    for(int stride = 1 ; stride < blockDim.x ; stride *= 2){
        if(tid % (2 * stride) == 0 && tid + stride < blockDim.x){
            smem[tid] += smem[tid + stride];
        }
        __syncthreads();
    }

    if(tid == 0){
        out[blockIdx.x] = smem[0];
    }
}
```

- 每个 Block 只处理 `blockDim.x` 个元素，所以 Grid 巨大（`N/256 = 524288` 个 Block）
- `tid % (2 * stride) == 0` 让同一 Warp 内只有一部分线程满足条件：
  - `stride = 1`：只有偶数 tid 工作 → 利用率 50%
  - `stride = 2`：每 4 个线程 1 个 → 25%
  - 之后越来越差
- 实测 **92.95 GB/s**，四类瓶颈里的「计算效率低」占了主导

V1：交错寻址（strided index）

```
// V1: 采用 stride index 方式，消除 Warp Divergence
for(int stride = 1 ; stride < blockDim.x ; stride *= 2){
    int idx = tid * 2 * stride;
    if(idx < blockDim.x)
        smem[idx] += smem[idx + stride];
    __syncthreads();
}
```

把判断从「谁满足 `tid % 2s == 0` 」换成「谁算出来的下标还在范围内」，活跃线程因此变成**连续的低编号线程**，Warp 要么整体进入、要么整体跳过：

- `s = 1`： `idx = 2·tid` ，活跃 tid 0~127，恰好 4 个完整 Warp → 无分化
- `s = 2`：活跃 tid 0~63，2 个完整 Warp → 无分化
- `s = 4`：活跃 tid 0~31，1 个完整 Warp → 无分化
- `s ≥ 8`：活跃线程不足 32 个，只有 Warp 0 内部有分化，开销很小

遗留问题：Bank Conflict。 以 `s = 1` 为例，tid 0 访问 `smem[0]/smem[1]` ，tid 16 访问 `smem[32]/smem[33]` ， `smem[0]` 和 `smem[32]` 同属 Bank 0 → 2 路冲突； `s = 2` 时 4 路， `s = 4` 时 8 路。

实测 **128.71 GB/s**（1.39x）。

V2：步长反转 —— 一次解决 Divergence + Bank Conflict

```
// V2: 步长从大到小，同时消除 Warp Divergence 与 Bank Conflict
for(int s = blockDim.x / 2 ; s > 0 ; s >>= 1){
    if(tid < s){
        smem[tid] += smem[tid + s];
    }
    __syncthreads();
}
```

`step = 128` 时：tid 0 访问 `smem[0], smem[128]` ，tid 1 访问 `smem[1], smem[129]` ，……，tid 31 访问 `smem[31], smem[159]` 。32 个线程正好覆盖 **Bank 0~31 的不同地址** → 无 Bank Conflict。

同时活跃线程永远是 `tid < s` 的连续低编号线程 → 整 Warp 工作或整 Warp 空闲 → 无 Warp Divergence。

实测 **160.68 GB/s**（1.73x）。

V3：每线程处理 2 个元素 —— 减少 idle 线程

V0~V2 中每个线程只负责 1 个元素，规约第一轮之后就有**一半线程彻底闲置**（它们唯一的贡献是搬运数据）。

```
int gid = (2 * blockDim.x) * blockIdx.x + threadIdx.x;

// 每个线程加载 2 个相距 blockDim.x 的元素并求和
float val = 0.0f;
if(gid < N)               val += in[gid];
if(gid + blockDim.x < N)  val += in[gid + blockDim.x];
smem[tid] = val;
__syncthreads();
```

对比项	V2（1 元素/线程）	V3（2 元素/线程）
每 Block 处理元素数	256	512
所需 Block 数	524,288	262,144
加载阶段计算	0 次加法	1 次加法/线程
规约轮数	8 轮	8 轮（不变）

Grid 配置要同步修改：`(N + 2*blockDim.x - 1) / (2*blockDim.x)`。

实测 **308.20 GB/s**（3.32x）。把 `V3.cu` 里被注释掉的 `gridSize = 1024` 打开、跑 Grid-Stride Loop 后达到 **578.47 GB/s** —— 这是本仓库单点收益最大的一处改动。

V4：展开最后一个 Warp

`s <= 32` 之后只剩 1 个 Warp 在工作，此时循环里的 `__syncthreads()` 完全多余（同 Warp 内同路径执行本身就是同步的）。

```
// 辅助函数：展开最后 32 个线程的规约
// warpReduce 中的 smem 必须声明为 volatile
// 防止编译器将中间结果缓存到寄存器，导致其他线程读取到旧值
__device__ void warpReduce(volatile float* smem, int tid) {
    smem[tid] += smem[tid + 32];
    smem[tid] += smem[tid + 16];
    smem[tid] += smem[tid +  8];
    smem[tid] += smem[tid +  4];
    smem[tid] += smem[tid +  2];
    smem[tid] += smem[tid +  1];
}

for(int s = blockDim.x / 2 ; s > 32 ; s >>= 1){ ... __syncthreads(); }

if(tid < 32){ warpReduce(smem, tid); }   // 无需 __syncthreads()
```

V3 有 8 轮循环（128, 64, 32, 16, 8, 4, 2, 1），V4 把后 5 轮（ `s ≤ 32` ）展开成直线代码，省掉 5 次 `__syncthreads()` 。

实测 **455.22 GB/s**（4.90x）。

V5：模板参数实现完全循环展开

```
template <int BLOCK_SIZE>
__global__ void reduce_v5(const float* in, float* out, int N){
    int gid = (2 * BLOCK_SIZE) * blockIdx.x + threadIdx.x;
    ...
    if (BLOCK_SIZE >= 512) { if (tid < 256) smem[tid] += smem[tid + 256]; __syncthreads(); }
    if (BLOCK_SIZE >= 256) { if (tid < 128) smem[tid] += smem[tid + 128]; __syncthreads(); }
    if (BLOCK_SIZE >= 128) { if (tid <  64) smem[tid] += smem[tid +  64]; __syncthreads(); }
    if (tid < 32) {
        volatile float* vsmem = smem;
        if (BLOCK_SIZE >= 64) vsmem[tid] += vsmem[tid + 32];
        vsmem[tid] += vsmem[tid + 16];
        ...
    }
}

// 调用端按 block 大小选择实例化
switch (blockSize) {
    case 512: reduce_v5<512><<<grid, 512, 512*sizeof(float)>>>(d_in, d_out, N); break;
    case 256: reduce_v5<256><<<grid, 256, 256*sizeof(float)>>>(d_in, d_out, N); break;
    case 128: reduce_v5<128><<<grid, 128, 128*sizeof(float)>>>(d_in, d_out, N); break;
}
```

`BLOCK_SIZE` 在编译期已知，不满足条件的分支（如 `BLOCK_SIZE >= 512` ）会被直接删除，最终生成无循环跳转、无死代码的直线指令序列。

实测 **469.50 GB/s**（5.05x）——提升有限，说明此时瓶颈已经不在循环开销上。

V6：Warp Shuffle 替代 Shared Memory

```
__device__ float warp_reduce_sum(float val){
    // 每次将右半边的值加到左半边
    for(int offset = 16 ; offset > 0 ; offset >>= 1)
        val += __shfl_down_sync(0xffffffff, val, offset);
    return val;   // lane 0 持有结果
}
```

Warp 内 32 个线程可以直接做**寄存器级通信**，不经过 Shared Memory：

- 不需要地址计算

- 不会有 Bank Conflict
- 延迟更低

但 Shuffle 只能在单个 Warp 内进行，所以 Block 级别要做**两级规约**：

1. **第一级**：每个 Warp 内 `__shfl_down_sync` 把 32 个值规约成 1 个（仅 lane 0 有效）
2. **第二级**：lane 0 写入 `__shared__ float warp_results[32]` → `__syncthreads()` → Warp 0 再 shuffle 一次

```
__shared__ float warp_results[32];   // 最多 32 个 Warp(1024/32)
int lane_id = tid % 32;
int warp_id = tid / 32;
if(lane_id == 0) warp_results[warp_id] = val;
__syncthreads();

int num_warps = blockDim.x / 32;
if(warp_id == 0){
    val = (lane_id < num_warps) ? warp_results[lane_id] : 0.0f;
    val = warp_reduce_sum(val);
}
if(tid == 0) out[blockIdx.x] = val;
```

实测 **522.54 GB/s**（5.62x）。

V7：Grid-Stride Loop + Warp Shuffle

```
int gid    = blockDim.x * blockIdx.x + tid;
int stride = blockDim.x * gridDim.x;

float val = 0.0f;
for(int idx = gid ; idx < N ; idx += stride)
    val += in[idx];

val = warp_reduce_sum(val);   // 后面接 V6 的两级规约
```

固定 Grid 大小，让每个线程以 `blockDim.x * gridDim.x` 为步长循环处理多段数据：

- Grid 规模与数据规模解耦，可以设成「刚好填满 GPU」
- 每个线程多干活，摊销 Kernel 启动与规约的固定开销
- 数组超过 Grid 上限时同样适用

实测 **581.67 GB/s**（6.26x），本仓库最优。

> 注意： `V7.cu` 中 `if(gridSize > 1024) gridSize = 2048;` 会让当前代码打印 `Grid size = 2048`，而文件头记录的是 Grid = 1024 时的跑分。从上面的扫描表看，1024 与 2048 属于噪声级差异。

V8：V7 + float4 向量化加载

```
const float4* in4 = reinterpret_cast<const float4*>(in);
int N4 = N / 4;

for(int idx = gid ; idx < N4 ; idx += stride){
    float4 data = in4[idx];
    val += data.x;  val += data.y;  val += data.z;  val += data.w;
}

// 处理 N 不是 4 的倍数时的尾部元素
int tail_start = N4 * 4;
for(int idx = tail_start + gid ; idx < N ; idx += stride)
    val += in[idx];
```

每条 `ld.global.v4.f32` 搬运 16 Byte，是 `ld.global.f32` 的 4 倍；在相同循环次数下处理的数据量翻 4 倍，等效减少循环次数与地址计算，并提升 ILP。

实测 **578.11 GB/s**，与 V7 基本持平 —— 说明此时已经撞到该卡的**带宽墙**（DRAM bound）。按照 Roofline 的分析，带宽饱和之后再减少指令数/循环开销都不会有收益。这也反过来验证了「Reduce 优化的本质是提高带宽利用率」。

四、关键知识点

Warp Divergence（Warp 分化）

- GPU 以 32 线程为一个 Warp 调度，Warp 内所有线程同一时刻执行同一条指令
- 分支条件让 Warp 内线程走不同路径时，硬件只能把两条路径**串行执行**，吞吐下降
- V0 的 `tid % (2 * stride) == 0` 是最典型的反面教材
- 消除思路：让**连续的低编号线程**成为活跃线程，使整个 Warp 一起进入 / 一起跳过

Shared Memory Bank Conflict

- Shared Memory 被划分为 32 个 Bank（32-bit 模式，每个 Bank 宽 4 Byte），地址按 4 Byte 依次循环映射：Bank 0, 1, …, 31, 0, 1, …
- 同一 Warp 内 32 个线程访问 32 个**不同 Bank** → 一次完成，无冲突
- 多个线程访问**同一 Bank 的不同地址** → 访问串行化，即 N 路 Bank Conflict
- 多个线程访问**同一地址** → 硬件广播，**不冲突**
- V1 的 `smem[idx]`（`idx = 2·s·tid`）让 Bank 号成倍跳，产生 2/4/8 路冲突；V2 的步长反转天然规避

Warp 内隐式同步 vs `__syncthreads()`

- `__syncthreads()` 是 **Block 级**屏障，代价较高
- 同一 Warp 内线程走同一路径时天然锁步，`step ≤ 32` 时不再需要屏障
- 但用 Shared Memory 在 Warp 内传递数据时仍需 `volatile` 或 shuffle，防止编译器把中间结果缓存进寄存器

`__shfl_down_sync(mask, var, delta)`

- 每个线程读取 `lane_id + delta` 处线程的**寄存器值**
- `mask = 0xffffffff` 表示 Warp 内 32 个 lane 全部参与
- 不经 Shared Memory → 无地址计算、无 Bank Conflict、延迟更低
- 只有 Warp 内通信能力 → 必须配合两级规约

Grid-Stride Loop

```
for (int idx = gid; idx < N; idx += blockDim.x * gridDim.x)
```

- Grid 与数据规模解耦，可固定为「SM 数 × 4」之类的合适值
- 每个线程处理多段数据，摊销启动与规约的固定开销，也避免尾部 Block 空转
- 超大数组（超过 Grid 上限）同样适用

float4 向量化加载

- 地址需 16 Byte 对齐（`cudaMalloc` 保证 256 Byte 对齐，天然满足）
- `N` 不是 4 的倍数时要有 tail 处理
- 收益前提是**还没有打满带宽**；一旦带宽饱和，收益为零（本仓库 V8 即如此）

Occupancy 提醒

- Occupancy = 当前活跃线程数 / 硬件可执行的最大线程数
- 100% Occupancy **不必然**更快：寄存器被人为压低会导致溢出到 local memory（实际在显存上），反而变慢
- Reduce 计算强度低，通常不需要刻意追求满 Occupancy

与文档的版本编号对照

文档把「float4 + Grid Stride Loop」合在 V7；本仓库拆成了两版，所以编号不完全对齐：

文档	本仓库
V0 ~ V6	V0 ~ V6（一一对应）
V7（float4 + Grid-Stride + Shuffle）	V8；本仓库 V7 是去掉 float4 的版本

五、与文档（A100）结果对照

文档在 A100 80GB SXM4 上的结果（理论带宽 2039 GB/s）：

版本	V0	V1	V2	V3	V4	V5	V6	V7
带宽利用率	~15%	~32%	~40%	~45%	~52%	~62%	~72%	~85%
带宽 (GB/s)	~300	~653	~816	~918	~1060	~1265	~1468	~1733
相对速度	1.0x	2.1x	2.7x	3.0x	3.5x	4.1x	4.8x	5.7x

本机（环境里 `nvidia-smi` 无法连接驱动，数据取自各 `.cu` 文件头注释）：92.95 → 581.67 GB/s，**整体 6.26x**，优化趋势与文档完全一致；但因为硬件不同，绝对值和「利用率」不能直接照搬。以 V7/V8 的 ~580 GB/s 作为本卡实测饱和点即可。

结论：

- 本机收益最大的是 **V2 → V3** 与 **V3 引入 Grid-Stride Loop**（308 → 578 GB/s），即「每线程多干活 + 减少 Block 数 + 固定 Grid」
- V4 / V5 的同步与循环开销优化在本机收益较小
- V8 的 float4 没有超过 V7，说明已到带宽墙
- 生产环境直接使用 NVIDIA CUB 的 `cub::DeviceReduce::Sum`，通常可跑到 90% 以上带宽利用率

六、自我检验清单

- [ ] 能解释 Warp Divergence 的成因，以及 V1（交错寻址）和 V2（步长反转）分别如何减少/消除它
- [ ] 能描述 Shared Memory Bank Conflict，并说明 V2 为什么能同时消除它
- [ ] 能解释 V3「每线程处理 2 个元素」的性能收益来源（Block 数减半、闲置线程减少）
- [ ] 能解释 V4 为什么在最后一个 Warp 中可以省去 `__syncthreads()`，以及为什么 `smem` 要加 `volatile`
- [ ] 能解释模板参数 `template <int BLOCK_SIZE>` 为什么能提升性能
- [ ] 能手写 `__shfl_down_sync` 的 Warp 内规约
- [ ] 能说清两级规约（Warp 内 + Warp 间）的完整流程
- [ ] 能说明 `float4` 向量化加载的优势与前提（对齐、tail、带宽是否已饱和）
- [ ] 能描述 Grid-Stride Loop 的工作方式及适用场景
- [ ] 能用 Nsight Compute 的 Memory Throughput 指标验证各版本的带宽利用率

七、本目录文件索引

文件	说明
`V0.cu`	朴素树形规约
`V1.cu`	交错寻址，减少 Warp Divergence
`V2.cu`	步长反转，消除 Bank Conflict
`V3.cu`	每线程 2 元素（含被注释的 Grid-Stride 版本）
`V4.cu`	展开最后一个 Warp（`warpReduce` + `volatile`）
`V5.cu`	模板参数完全循环展开
`V6.cu`	Warp Shuffle 两级规约
`V7.cu`	Grid-Stride Loop + Warp Shuffle
`V8.cu`	V7 + float4 向量化加载（含 tail 处理）
`Test.cu`	V7 / V8 对比 Benchmark，Grid = 128~8192 扫描，Warmup 5 / Iter 100

文件	说明
`Test_V7.cu`	V7 的 Grid 扫描
`V3_report.ncu-rep` 、 `V7_report.ncu-rep`	Nsight Compute profile 报告

八、参考资料

- AIInfraGuide — 3.1 CUDA Reduce 算子优化
- NV 官方教程 / CUDA C++ Programming Guide - Warp Shuffle Functions
- Mark Harris, NVIDIA — Optimizing Parallel Reduction in CUDA
- 深入浅出 GPU 优化系列：reduce 优化
- NVIDIA CUB Library — `cub::DeviceReduce`
- CUDA C++ Best Practices Guide — Memory Optimizations
- NVIDIA Nsight Compute Documentation